



Vijay Academy Presents



Savitribai Phule Pune University (SPPU)

Web Technology (WT)

TE Computer Engineering (2019 Pattern)

One Shot Lecture

Unit 5. Server Side Scripting Languages

What You Get : Strategy For Exam :

- Simple Explanation
- PPT + Notes
- PYQs + Most IMP Questions



Subscribe The Vijay Academy For Free Engineering Content

Unit 5. Server Side Scripting Languages



Syllabus Topics :

PHP Topics:

1. Introduction to PHP & Uses of PHP
2. General Syntactic Characteristics of PHP
3. Primitives, Operations and Expressions in PHP
4. Output in PHP
5. Control Statements in PHP
6. Arrays in PHP
7. Functions in PHP
8. Pattern Matching in PHP
9. Form Handling in PHP
10. Files in PHP
11. Cookies in PHP
12. Session Tracking in PHP
13. Using MySQL with PHP
14. WAP and WML

ASP.NET Topics:

15. Overview of .NET Framework
16. Overview of C#
17. Introduction to ASP.NET
18. ASP.NET Controls
19. Web Services in ASP.NET

Node JS:

20. Overview of Node JS

1. Introduction to PHP & Uses of PHP

What is PHP ?

PHP = Hypertext Preprocessor It is a **server-side scripting language** used to build dynamic websites.

Code runs on the **server**.

Output sent to browser as **HTML**.

Free & Open Source.

Works with **MySQL**.

Example :

```
<?php
```

```
    echo "Hello, Welcome to PHP!";
```

```
?>
```

Output:

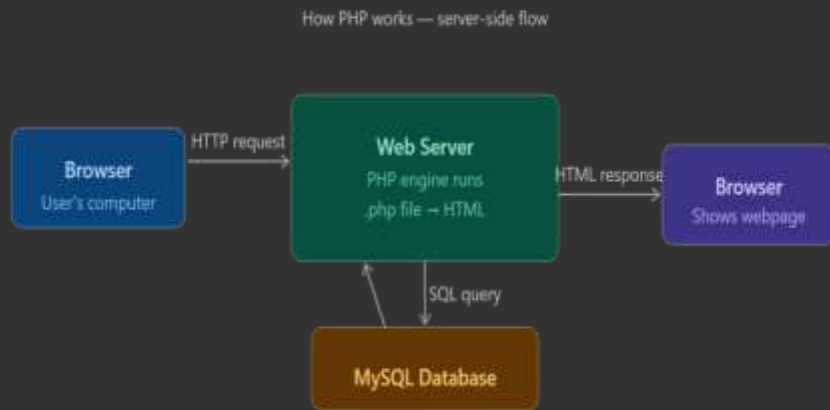
Hello, Welcome to PHP!

Explanation:

<?php → starts PHP code,

echo → prints output,

?> → ends PHP code





1. Introduction to PHP & Uses of PHP

Key Points

PHP stands for **Hypertext Preprocessor**

It is **open source** and **free** to use

PHP files have extension **.php**

Output is sent to browser as **plain HTML**

PHP can connect to **databases** like MySQL

PHP works on **Windows, Linux, Mac**

Uses of PHP :

Building **dynamic web pages**

Form handling (login, registration)

Database operations (MySQL)

Cookies & Sessions management

File handling (read/write files)

User **authentication systems**

E-commerce websites (WordPress, Magento use PHP)

2. General Syntactic Characteristics of PHP

Syntax means the **rules of writing code** in a language. PHP has its own set of rules you must follow.

Syntax Rules of PHP

1. **PHP tags** PHP code always starts and ends with special tags:

```
<?php  
    // your code here  
?>
```

2. **Statements end with semicolon ;** Every line/statement must end with ;

```
<?php  
    echo "Hello";  
    $x = 10;  
?>
```

2. General Syntactic Characteristics of PHP

3. Variables start with \$ sign

```
<?php
$name = "Vijay";
$age = 21;
?>
```

4. PHP is case-sensitive for variables \$name and \$Name are **different** variables. But function names like echo, ECHO are same.

5. Comments in PHP

```
<?php
// This is single line comment
# This is also single line comment
/* This is
multi-line comment */
?>
```

2. General Syntactic Characteristics of PHP

6. Strings use single or double quotes

```
<?php
```

```
    $a = "Hello World"; // double quotes
```

```
    $b = 'Hello World'; // single quotes
```

```
?>
```

7. echo and print for output

```
<?php
```

```
    echo "Hello"; // faster, no return value
```

```
    print "Hello"; // returns 1, slightly slower
```

```
?>
```

3. Primitives, Operations and Expressions in PHP

PHP Data Types (Primitives) :

1. Integer Whole numbers (no decimal)

```
<?php  
$age = 21;  
$marks = 95;  
?>
```

2. Float (Double) Numbers with decimal point

```
<?php  
$price = 99.99;  
$pi = 3.14;  
?>
```

3. String Text inside quotes

```
<?php  
$name = "Vijay";  
$city = 'Nashik';  
?>
```

4. Boolean Only two values — true or false

```
<?php  
$isLoggedIn = true;  
$isAdmin = false;  
?>
```

5. NULL Variable with no value

```
<?php  
$x = NULL;  
?>
```

3. Primitives, Operations and Expressions in PHP

Operations in PHP

Arithmetic Operators:

```
$a = 10; $b = 3;  
echo $a + $b;      // 13  
echo $a - $b;      // 7  
echo $a * $b;      // 30  
echo $a / $b;      // 3.33  
echo $a % $b;      // 1 (remainder)
```

Comparison Operators :

```
$a == $b           // equal  
$a != $b           // not equal  
$a > $b            // greater than  
$a < $b            // less than
```

Logical Operators:

```
$a && $b           // AND  
$a || $b           // OR  
!$a                // NOT
```

Expressions :

Expression = anything that has a **value**.

```
$x = 5 + 3;        // expression gives 8  
$y = $x * 2;       // expression gives 16  
$z = ($x > $y);    // expression gives false
```

4. Output in PHP

Output means **displaying data** on the web page.

Ways to Display Output in PHP

1. **echo** Most commonly used. Can output one or more strings.

```
<?php
echo "Hello World";
echo "My name is " . "Vijay";
echo "<br>";           // HTML line break
echo "Age: ", 21;      // multiple values with comma
?>
```

2. **print** Similar to echo. Can only output one string.

```
<?php
print "Hello World";
print "<br>";
print "Welcome to PHP";
?>
```

5. Control Statements in PHP

What are Control Statements?

Control statements **control the flow** of a program. They decide **which code runs** and **how many times** it runs.

Types of Control Statements :

1. if statement

Run code **only if condition** is true.

```
<?php
$marks = 75;
if($marks >= 40){
    echo "Pass";
}
?>
```

Output: Pass

2. if-else statement

Run one block if true, **another if false**.

```
<?php
$marks = 30;
if($marks >= 40){
    echo "Pass";
} else {
    echo "Fail";
}
?>
```

Output: Fail

3. if-elseif-else statement

Check **multiple conditions**.

```
<?php
$marks = 85;
if($marks >= 75){
    echo "Distinction";
} elseif($marks >= 60){
    echo "First Class";
} elseif($marks >= 40){
    echo "Pass";
} else {
    echo "Fail";
}
?>
```

Output: Distinction

4. switch statement

Check one variable against **many values**.

```
<?php
$day = 2;
switch($day){
    case 1:
        echo "Monday";
        break;
    case 2:
        echo "Tuesday";
        break;
    case 3:
        echo "Wednesday";
        break;
    default:
        echo "Other day";
}
?>
```

Output: Tuesday

5. while loop

Repeat code **while condition is true**.

```
<?php
$i = 1;
while($i <= 5){
    echo $i . " ";
    $i++;
}
?>
```

Output: 1 2 3 4 5

6. do-while loop

Runs code **at least once**, then checks condition.

```
<?php
$i = 1;
do {
    echo $i . " ";
    $i++;
} while($i <= 5);
?>
```

Output: 1 2 3 4 5

7. for loop

Use when you know **exact number of times** to repeat.

```
<?php
for($i = 1; $i <= 5; $i++){
    echo $i . " ";
}
?>
```

Output: 1 2 3 4 5

8. foreach loop

Used specially for **arrays**.

```
<?php
$fruits = array("Apple", "Mango", "Banana");
foreach($fruits as $fruit){
    echo $fruit . "<br>";
}
?>
```

Output:

Apple
Mango
Banana

6. Arrays in PHP

What is an Array?

An array is a **single variable that stores multiple values** at once. Instead of making 5 variables for 5 students, you make **one array**.

Example :

```
<?php
```

```
// Without array (bad way)
```

```
$s1 = "Vijay";
```

```
$s2 = "Rahul";
```

```
$s3 = "Priya";
```

```
// With array (good way)
```

```
$students = array("Vijay", "Rahul", "Priya");
```

```
?>
```

6. Arrays in PHP

Types of Arrays in PHP :

1. Indexed Array

Items stored with **numeric index** starting from 0.

```
<?php
```

```
$fruits = array("Apple", "Mango", "Banana");
```

```
echo $fruits[0]; // Apple
```

```
echo $fruits[1]; // Mango
```

```
echo $fruits[2]; // Banana
```

```
?>
```

Output:

Apple

Mango

Banana

Types of arrays in PHP

Indexed array numeric keys



Associative array named keys



Multidimensional array array inside array — like a table

Row	Col [0] Name	Col [1] Age	Col [2] City
[0]	Vijay	21	Pune
[1]	Rahul	22	Mumbai
[2]	Priya	20	Nashik



6. Arrays in PHP

2. Associative Array

Items stored with **named keys** (key => value pairs).

```
<?php
```

```
$student = array(  
    "name" => "Vijay",  
    "age"  => 21,  
    "city" => "Pune"  
);
```

```
echo $student["name"]; // Vijay  
echo $student["age"];  // 21  
echo $student["city"]; // Pune  
?>
```

Output:

name: Vijay
age: 21
city: Pune

3. Multidimensional Array

Array **inside another array**.

Like a table with rows and columns.

```
<?php
```

```
$students = array(  
    array("Vijay", 21, "Pune"),  
    array("Rahul", 22, "Mumbai"),  
    array("Priya", 20, "Nashik")  
);
```

```
// Access specific value
```

```
echo $students[0][0]; // Vijay  
echo $students[1][2]; // Mumbai  
?>
```

Output:

Vijay
Mumbai

7. Functions in PHP

What is a Function?

A function is a **block of code** that does a specific task.

You write it **once** and can **use it many times**.

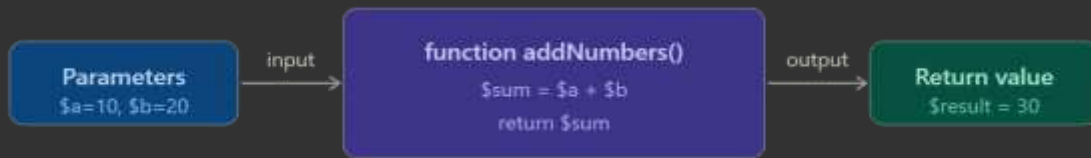
Why Use Functions?

Avoid writing same code again and again.

Makes code clean and organized.

Easy to fix/update — change only one place.

How a PHP function works



Call: addNumbers(10, 20)

Result: echo \$result → 30

Types: built-in · user-defined · with params · with return · default params · recursive



Types of Functions in PHP

1. Built-in Functions

PHP has **ready-made functions** you can directly use.

```
<?php
echo strlen("Vijay");           // 5 (string length)
echo strtoupper("hello");       // HELLO
echo strtolower("HELLO");       // hello
echo abs(-10);                  // 10 (absolute value)
echo sqrt(16);                  // 4
?>
```

2. User-defined Functions

Functions **you create yourself**.

Basic syntax:

```
<?php
// Define function
function greet(){
    echo "Hello! Welcome!";
}

// Call function
greet();
?>
```

Output: Hello! Welcome!



Types of Functions in PHP

3. Function with Parameters

Pass values into function.

```
<?php
function greetStudent($name){
    echo "Hello " . $name . "!"<br>";
}

greetStudent("Vijay");
greetStudent("Rahul");
greetStudent("Priya");
?>
```

Output:

Hello Vijay!
Hello Rahul!
Hello Priya!

4. Function with Return Value

Function calculates and returns a result.

```
<?php
function addNumbers($a, $b){
    $sum = $a + $b;
    return $sum;
}

$result = addNumbers(10, 20);
echo "Sum = " . $result;
?>
```

Output: Sum = 30



Types of Functions in PHP

5. Function with Default Parameters

If no value passed, use **default value**.

```
<?php
function greet($name = "Student"){
    echo "Hello " . $name . "!<br>";
}

greet("Vijay"); // Hello Vijay!
greet();         // Hello Student! (uses default)
?>
```

6. Recursive Function

Function that **calls itself**.

Useful for problems like factorial.

```
<?php
function factorial($n){
    if($n == 1){
        return 1;
    }
    return $n * factorial($n - 1);
}

echo factorial(5); // 120
?>
```

How it works:

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120
```

8. Pattern Matching in PHP

What is Pattern Matching?

Pattern matching means **searching for a specific pattern** inside a string.

In PHP, this is done using **Regular Expressions (Regex)**.

What is a Regular Expression?

A regular expression is a **special sequence of characters** that defines a search pattern.

Example:

Find all **numbers** in a string

Check if **email is valid**

Check if **phone number** format is correct

Common Regex Patterns

Pattern	Meaning	Example Match
[0-9]	Any digit	3, 7, 0
[a-z]	Any lowercase letter	a, b, z
[A-Z]	Any uppercase letter	A, B, Z
[a-zA-Z]	Any letter	a, B, Z
.	Any character	a, 3, #

9. Form Handling in PHP

What is Form Handling?

Form handling means **collecting data from HTML form** and processing it using PHP.

Example — Login page, Registration form, Contact form.

Two Methods to Send Form Data

1. GET Method

Data sent in **URL**

Visible in browser address bar

Used for **search forms**

Less secure

2. POST Method

Data sent **hidden** (not in URL)

Not visible in address bar

Used for **login, registration**

More secure

10. Files in PHP

What is File Handling?

File handling means **reading, writing and managing files** on the server using PHP.

Example — saving user data to a text file, reading a log file, writing results to a file.

Common File Functions in PHP :

`fopen()` → open file (must do first)

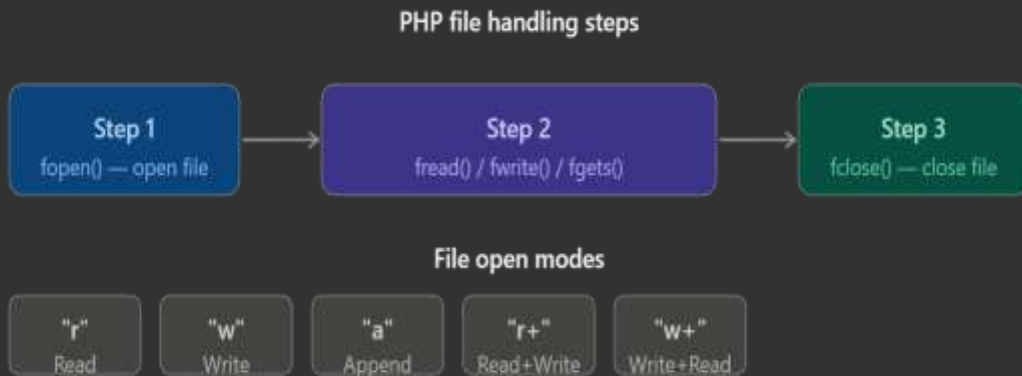
`fwrite()` → write data to file

`fread()` → read data from file

`fgets()` → read file line by line

`fclose()` → close file (must do last)

`file_exists()` → check if file exists



Always `fclose()` after use — `feof()` checks end of file — `fgets()` reads line by line

11. Cookies in PHP

What is a Cookie?

A cookie is a **small piece of data** stored in the **user's browser** by the server.

Why Use Cookies?

Remember user's **login info**

Remember user **preferences** (like dark mode)

Track user **visits**

Keep user **logged in** even after closing browser.

Key Points :

`setcookie("name", "value", expiry)` → create cookie

`$_COOKIE["name"]` → read cookie

Delete cookie → set expiry in past

`isset($_COOKIE["name"])` → check if cookie exists

Cookies stored **client side** — not fully secure

Max size = **4KB**.

How Cookies Work :

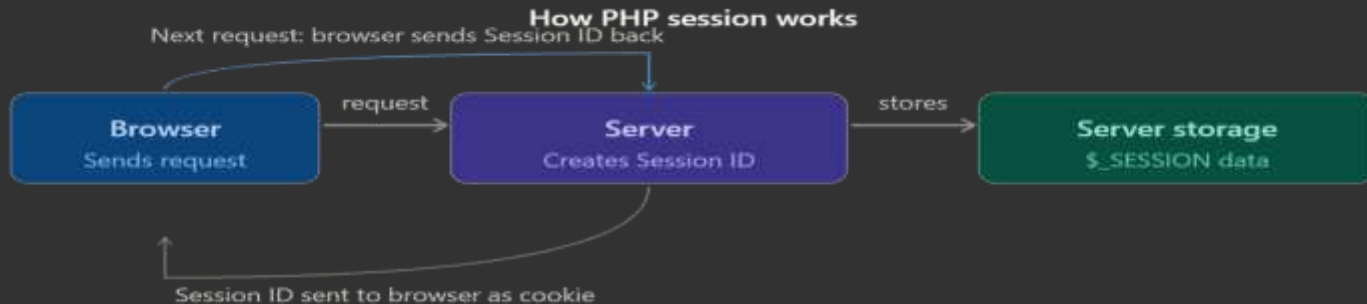


12. Session Tracking in PHP

What is a Session?

A session is a way to **store user information on the server** and use it across multiple pages.

How Cookies Work :



Key Points :

`session_start()` → must be on **every page, first line.**

`$_SESSION["key"] = value` → set session variable.

`$_SESSION["key"]` → read session variable.

`unset($_SESSION["key"])` → remove one variable.

`session_destroy()` → destroy all session data (logout)

Session ends when **browser closes.**

More secure than cookies — data on server.

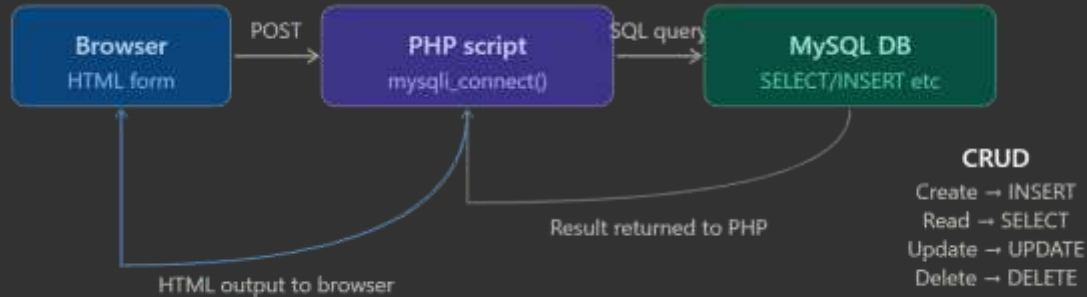
13. Using MySQL with PHP

What is MySQL with PHP?

MySQL is a **database** used to store data. PHP connects to MySQL to **store, fetch, update and delete** data.

Example — storing student records, login data, product info.

PHP with MySQL — flow





13. Using MySQL with PHP

Steps to Connect PHP with MySQL :

Step 1 — Connect to Database

Connect PHP to MySQL using `mysqli_connect()`.

```
$conn = mysqli_connect("localhost", "root", "", "mydb");
```

Step 2 — Write SQL Query

Write your SQL command as a string in PHP.

```
$sql = "SELECT * FROM students";
```

Step 3 — Execute Query

Run the query using `mysqli_query()`.

```
$result = mysqli_query($conn, $sql);
```

Step 4 — Fetch Results

Get data row by row using `mysqli_fetch_assoc()`.

```
$row = mysqli_fetch_assoc($result); then use $row["name"], $row["age"] etc.
```

Step 5 — Display Data

Use `echo` to show data on webpage.

```
echo $row["name"];
```

Step 6 — Close Connection

Always close connection after use.

```
mysqli_close($conn);
```

14. WAP and WML

What is WAP?

WAP stands for **Wireless Application Protocol**.

It is a standard that allows **mobile phones** to access internet and web content.

What is WML?

WML stands for **Wireless Markup Language**.

It is the language used to create pages for **WAP browsers** on mobile phones.

WML is to WAP what **HTML is to Web browsers**.

14. WAP and WML

WAP Architecture :



WAP Components

1. **WAP Browser** - Special browser on mobile phone that reads WML pages.
2. **WAP Gateway** - Most important component. Acts as a **bridge** between mobile network and internet. Converts WAP requests to HTTP and back.
3. **Web Server** Stores and serves WML pages to gateway.
4. **WAP Protocol Stack** Layers of WAP communication:
 - WAE** — Wireless Application Environment (top layer, WML lives here)
 - WSP** — Wireless Session Protocol
 - WTP** — Wireless Transaction Protocol
 - WTLS** — Wireless Transport Layer Security
 - WDP** — Wireless Datagram Protocol (bottom layer)

14. WAP and WML

What is WML?

WML pages are called **decks**. Each deck contains **cards**. Each card is like **one screen** on mobile.

WML Basic Structure :

```
<?xml version="1.0"?>
<wml>
  <card id="card1" title="Welcome">
    <p>Hello! Welcome to WAP!</p>
  </card>
</wml>
```

Explanation:

<wml> → root element (like <html>)

<card> → one screen on mobile

<p> → paragraph text.

14. WAP and WML

WML Important Elements :

1. <wml>

Root element of every WML page — like <html> in HTML, everything is written inside this.

2. <card>

Represents one screen on mobile — a deck can have many cards, each card = one page.

3. <p>

Used to display paragraph text on the mobile screen — like <p> in HTML.

4. <a href>

Used for navigation — clicking it takes user to another card or URL.

5. <input>

Used to take text input from user — like a text box on mobile screen.

6. <do>

Used to create a button or action — like a submit button on mobile.

7. <select>

Used to show a dropdown list — user can select one option from list.

15. Overview of .NET Framework

What is .NET Framework?

.NET Framework is a **software development platform** made by **Microsoft**.

It is used to build and run **Windows applications and web applications**.

Think of it like a **big toolbox** made by Microsoft — it gives you everything needed to build applications.

Key Points :

Made by **Microsoft**

Used to build **web, desktop, mobile** applications

Main language used = **C#** (C Sharp)

Runs on **Windows**

Current version = **.NET Core** (cross platform — Windows, Linux, Mac)

.NET Framework Architecture Diagram :

.NET Framework architecture



CLR is the heart of .NET — it manages code execution



Main Components of .NET Framework :

1. CLR — Common Language Runtime

It is the **heart of .NET framework**.

It manages execution of programs — memory, garbage collection, security.

Think of it like an **engine** that runs your code.

2. BCL — Base Class Library

A huge collection of **ready made classes and functions**.

Like string handling, file handling, math, collections etc.

You don't write everything from scratch — BCL gives it to you.

3. CIL — Common Intermediate Language

When you write C# code, it first compiles to **CIL** (intermediate code).

Then CLR converts CIL to **machine code** using JIT compiler.

4. JIT — Just In Time Compiler

Converts CIL code to **machine code at runtime**. Runs inside CLR.

5. ASP.NET

Part of .NET for building **web applications**. Used to create dynamic websites using C#.

16. Overview of C#

What is C#?

C# (pronounced **C Sharp**) is a **modern, object oriented programming language** made by **Microsoft**.

It is the **main language** used to build .NET applications.

Think of it like **Java but for Microsoft platform**.

Key Points :

Made by **Microsoft** (by Anders Hejlsberg)

Pronounced "**C Sharp**"

Object Oriented Language

Used with **.NET Framework**

Similar to **Java and C++**

Used for **web, desktop, games (Unity), mobile** apps



16. Overview of C#

Basic C# Program

```
using System;
```

```
class HelloWorld {  
    static void Main(string[] args) {  
        Console.WriteLine("Hello World!");  
    }  
}
```

Output: Hello World!

Explanation:

using System → import basic library

class HelloWorld → define a class

static void Main() → main method, program starts here

Console.WriteLine() → print output

16. Overview of C#

Features of C# :

Object Oriented — supports class, object, inheritance.

Garbage Collection — automatic memory management.

Simple Syntax — easy to learn.

Rich Library — BCL gives ready made functions.

C# vs Java

Feature	C#	Java
Made by	Microsoft	Sun/Oracle
Platform	.NET / Windows	JVM / Cross platform
Syntax	Similar to Java	Similar to C#
Used for	Windows, Web, Games	Android, Web, Enterprise
IDE	Visual Studio	Eclipse, IntelliJ
File extension	.cs	.java



17. Introduction to ASP.NET

What is ASP.NET?

ASP.NET stands for **Active Server Pages .NET**. It is a **web application framework** made by Microsoft.

Used to build **dynamic websites and web applications** using C#.

Think of it like **PHP** but for **Microsoft platform**.

Key Points :

Made by **Microsoft**

Part of **.NET Framework**

Used to build **dynamic web pages**

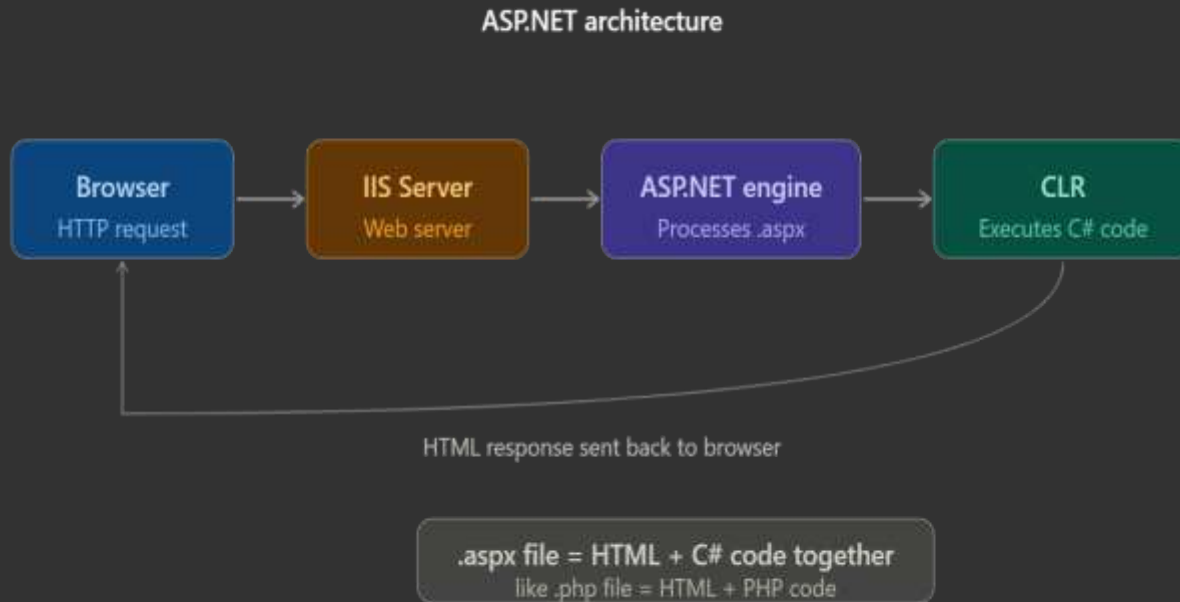
Main language = **C#**

Runs on **IIS (Internet Information Services)** web server

File extension = **.aspx**

17. Introduction to ASP.NET

ASP.NET Architecture Diagram :



18. ASP.NET Controls

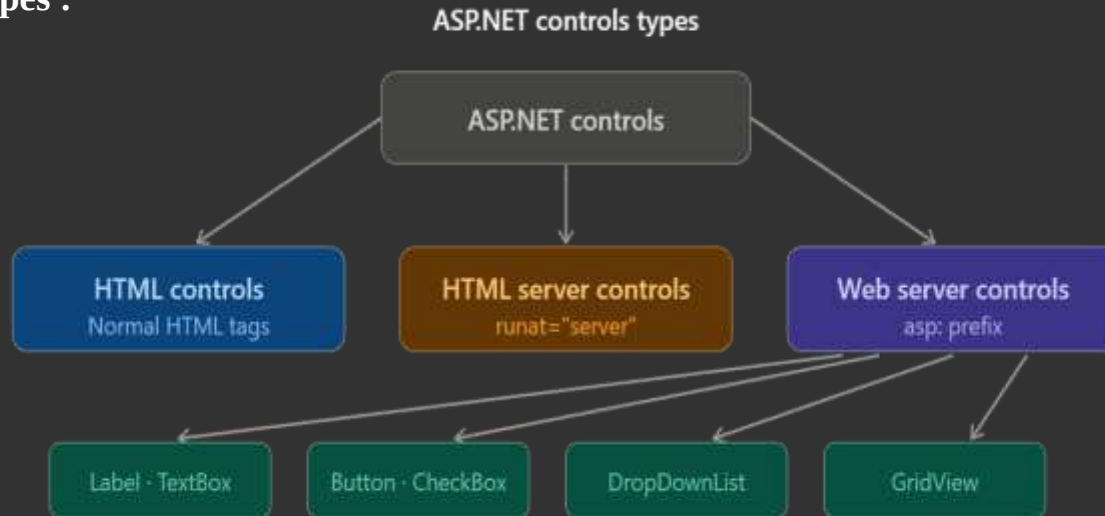
What are ASP.NET Controls?

ASP.NET Controls are **ready made UI components** that you can use directly in your web page.

Think of them like **building blocks** — instead of writing HTML manually, you use ready made controls.

Example — Button, TextBox, Label, DropDownList etc.

ASP.NET Controls Types :



19. Web Services in ASP.NET

What is a Web Service?

A Web Service is a way for **two applications to communicate** with each other over the internet.

Key Points

Web services allow **application to application** communication.

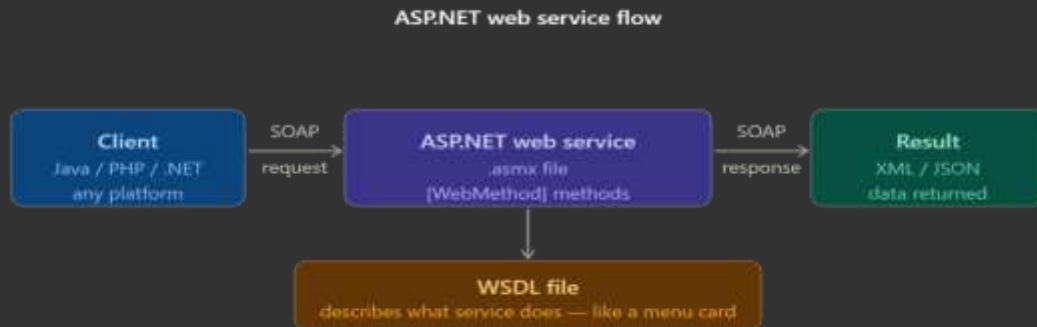
Works over **HTTP** protocol.

Data exchanged in **XML or JSON** format.

Platform independent — **Java app can talk to .NET app**

Language independent — any language can use it.

How Web Service Works :





20. Overview of Node JS

What is Node JS?

Node JS is a **JavaScript runtime environment** that allows you to run JavaScript on the **server side**.

Normally JavaScript runs only in browser. Node JS lets JavaScript run **outside the browser — on the server**.

Think of it like — **JavaScript got superpowers to run on server!**

Key Points :

Created by **Ryan Dahl** in **2009**

Built on **Chrome's V8 JavaScript engine**

Runs JavaScript on **server side**

Non-blocking — handles many requests at same time

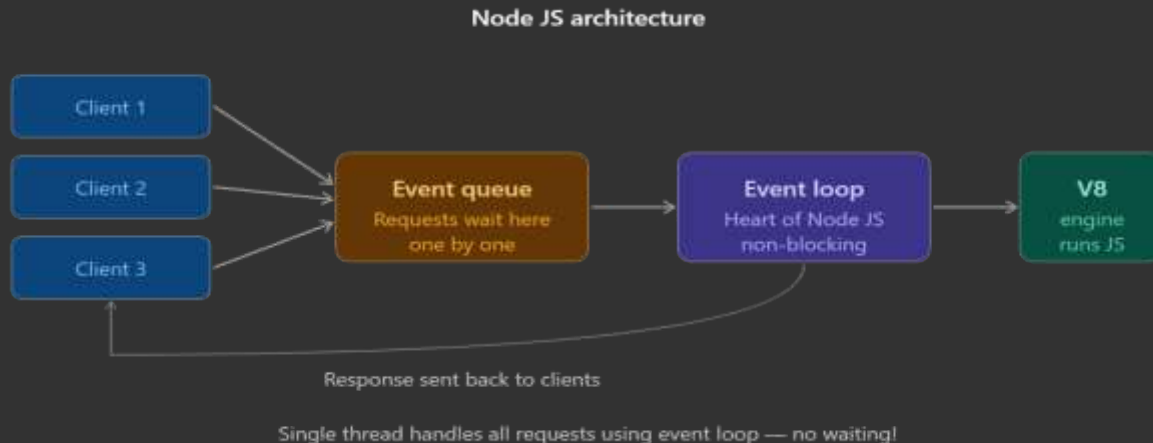
Uses **npm** (Node Package Manager) for libraries

Very fast and lightweight

Used by big companies — **Netflix, LinkedIn, Uber, PayPal**

20. Overview of Node JS

How Node JS Works :



Uses of Node JS :

Real time apps — chat apps, live notifications

REST APIs — backend for mobile/web apps

Streaming — Netflix uses Node JS

Microservices — small independent services

Command line tools



Thank You!



Like, Share & Subscribe

<https://youtube.com/@thevijayacademy?si=uYwlDuAxyZfLsiei>

Also join Telegram channel for updates and notes :

https://t.me/the_vijay_academy